Implementation and Benchmarking of ML-KEM-768 using PQM4 on STM32 F4 Discovery board

1. System Overview

This project implements and benchmarks **ML-KEM-768 (Kyber768)** on:

- **Board:** STM32 F4 Discovery

  The STM32F4 Discovery board features 1MB of Flash, and 192KB of RAM. Connecting the development to the host computer requires a mini-USB cable and a USB-TTL converter together with a 2-pin dupont / jumper cable.

- **Microcontroller:** STM32F407G-DISC1

  Schematics:
  https://www.st.com/resource/en/schematic_pack/mb997-f407vgt6-e01_schematic.pdf

- **Core:** ARM Cortex-M4F
- **Framework:** PQM4 (Post-Quantum Cryptography on Microcontrollers)

The goal is to:

- Build ML-KEM on Cortex-M4
- Flash the generated ELF binaries
- Capture serial output
- Benchmark performance
- Validate correctness

2. Hardware Requirements

2.1 Required Components

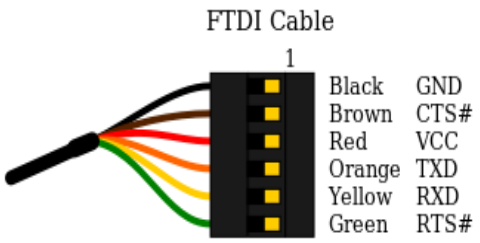
Component	Purpose
STM32 F4 Discovery	Target MCU
Micro-USB cable	Power + ST-LINK programming
2x jumper wires	UART connection (PA2 , PA3)

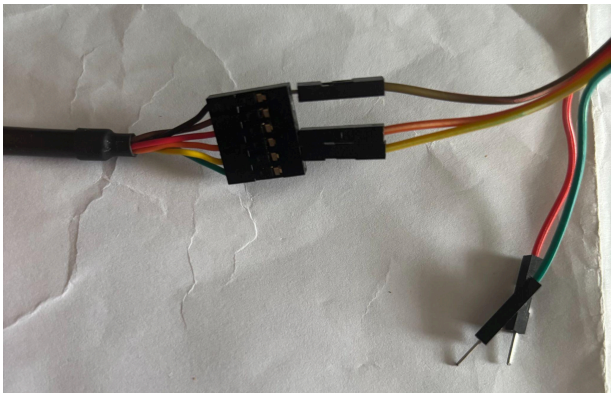
2.2 UART Pin Configuration

PQM4 uses **USART2** for output.

Function	Pin
USART2_TX	**PA2**
USART2_RX	**PA3**
GND	Any GND

Jumper Cable





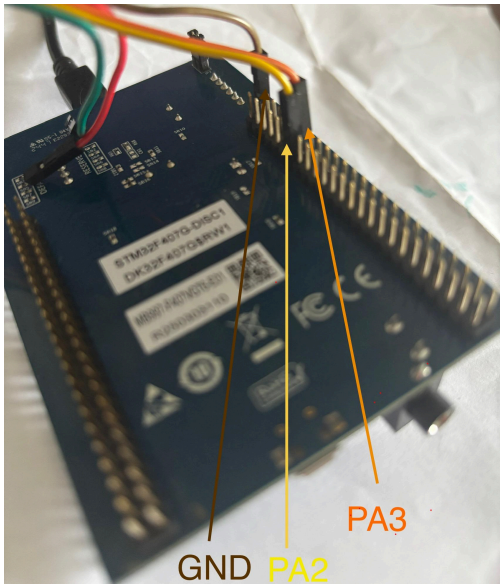
To see serial output:

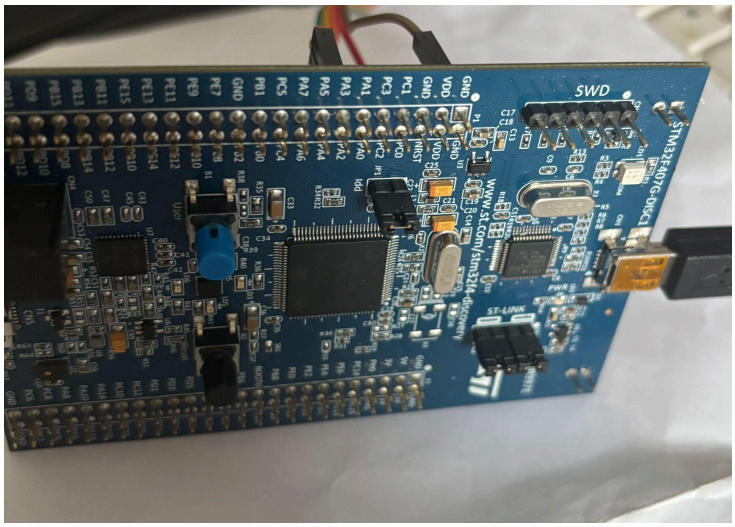
Connect PA2 , PA3 using a jumper cable to virtual COM port

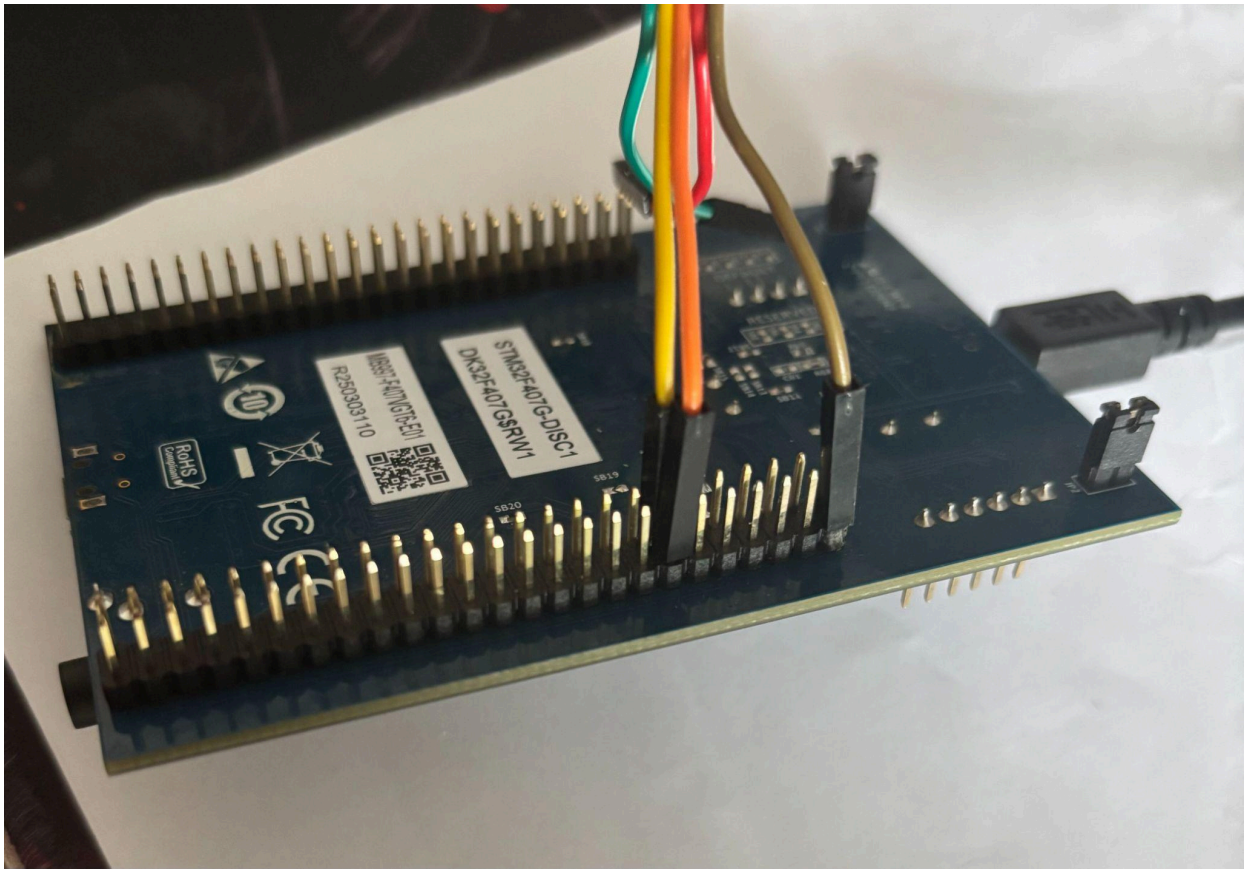
This allows TX output to be readable through serial output.

Connect like this:

Yellow -> PA2
Orange -> PA3
Brown/Black -> GND







2.3 Finding the serial device name

macOS

- List ports:

```
$ ls /dev/tty.*

/dev/tty.usbmodem11403    /dev/tty.usbserial-FT9M1XM6
```

We will be using `tty.usbserial-*` for viewing output.

Linux

- Often:
  - /dev/ttyUSB0 (USB-TTL)
  - /dev/ttyACM0

3. Software Requirements

This reflects exactly what worked in your setup.

3.1 Operating System

- macOS (tested successfully)
- Linux works similarly

3.2 Required Tools

- **git**

```
brew install git
```

- **make**

```
brew install make
```

- **A cross-compiler toolchain for ARM** (arm-none-eabi-gcc, arm-none-eabi-objcopy, arm-none-eabi-size)

```
brew install --cask gcc-arm-embedded
      arm-none-eabi-gcc --version
      If not in PATH:
export PATH=/Applications/ArmGNUToolchain/15.2.rel1/bin:$PATH
```

- **python3** (PQM4 scripts + helper tooling): version > Python 3.10

```
pip install pyserial tqdm
```

Or

```
pip3 install -r requirements.txt
```

- **st-flash / openocd / STM32CubeProgrammer** (one of these, depending how we flash)

  - For command-line flashing (st-link)

```
brew install stlink
```

- **STM32CubeIDE**

  Download Link:
  https://www.st.com/en/development-tools/stm32cubeide.html#st-get-software

  Used for:

  - Flashing ELF files
  - Debugging compiled binary files

  Version used successfully:

  - STM32CubeIDE 1.13+

- **WCHSerialPort tool**
  Download Link: https://www.wch-ic.com/downloads/category/30.html

  Used To:
  - Connect and listen to serial port. This can be used to view the output from MCU.
  - We can also use linux commands like screen.
  - or python code provided by pqm4 library (hostside/host_unidirectional.py) to view the output.